



UNIVERSIDAD DE GUAYAQUIL
BASE DE DATOS AVANZADO
FACULTAD DE CIENCIAS, MATEMÁTICAS Y FÍSICA
CARRERA: SOTFWARE

MANUAL TECNICO

GRUPO #2

INTEGRANTES:

- **RUBÉN QUIROGA PERALTA**
- **MARIO JACHO CEDEÑO**
- **GALO IZQUIERDO VALLEJO**
- **MIGUEL REYES GONZABAY**
- **DOUGLAS ORRALA VIOLÍN**
- **KLÉBER ARTEAGA BERRONES**
- **DIEGO ACOSTA SARCO**
- **ANDY GUTIERREZ PAREDES**
- **JONATHAN GELVEZ PIN**
- **NICOLÁS QUINDE ASPIAZU**

PARALELO: SOF-S-NO-6-7

Contenido

OBJETIVO DEL SISTEMA.....	4
Requisitos minimos:	4
Requisitos que deben tener Instalado en el equipo	4
DESCRIPCIÓN DE LAS FUNCIONALIDADES DEL SISTEMA:	5
INGRESO AL SISTEMA	5
Como se estructura el proyecto	5
1 bin.....	5
2. obj.....	6
3. sql	6
4. sql/system/	7
1. Archivos fuente .cs (clases de negocio)	8
2. Properties/	8
3. obj/ (igual a CData)	9
4. CNegocio.csproj	9
1. Subcarpeta bin.....	10
2. Subcarpeta obj.....	11
3. Subcarpeta Properties.....	11
4. Formularios del sistema (Vista)	12
Cada formulario contiene:	12
Funcionalidad del software.....	13
Funcionalidad de la base de datos	14
Interacción entre Capas del Sistema y la Base de Datos	14
Flujo de funcionamiento	15
Resultado se devuelve a la vista	15
Funcionalidad de los formularios y botones	15
Funcionalidad específica por sección de datos	16
Lógica de Ejecución en Base de Datos	17
Diagrama caso de usos	18
Diagrama entidad relación.....	19
Diagrama Base de datos	20
Mejoras Proyectadas a Futuro	21
Mejora de Interfaz Gráfica	21
Generación de Reportes y Exportación	21

Versión Web o Multiplataforma	21
MANEJO DE ERRORES	22
Reporte de Análisis de Excepción	22
Análisis	23
Causa Raíz	23
Recomendaciones	23
Conclusión:	24
Reporte de Error: MySql.Data.MySqlClient.MySqlException	25
Causas Probables	26

OBJETIVO DEL SISTEMA

El presente documento tiene como objetivo describir detalladamente la funcionalidad del Sistema de Gestión Administrativa para Taller de Reparación de Celulares. Este sistema está diseñado para optimizar y facilitar los procesos operativos del taller, incluyendo la gestión integral de órdenes de reparación, control y seguimiento de dispositivos, administración de clientes, así como la generación de facturas.

El lenguaje en el que esta hecho el programa es en C# y usa una base de datos SQL

Versión del programa 1.0.0.0

Requisitos minimos:

- Sistema operativo Windows 10
- Procesador Intel core celeron o superior
- 4 GB de Ram
- Espacio en el disco duro entre 2.2GB-5.5GB

Requisitos que deben tener Instalado en el equipo

- Visual Studio 2022
- Tener instalada la extensión de Desarrollo de escritorio con.NET(Visual Studio)
- Tener instalada Desarrollo de la interfaz de usuario de aplicaciones (Visual Studio)
- NET 6.0 Desktop Runtime (v6.0.36)
- MySQL 8.0 o superior
- HeidiSQL Cliente gráfico recomendado para administrar la base de datos MySQL.

DESCRIPCIÓN DE LAS FUNCIONALIDADES DEL SISTEMA:

INGRESO AL SISTEMA

Se ingresan las credenciales correspondientes en la siguiente ventana y se hace clic en aceptar para poder acceder al sistema.

Estructuración del software

Arquitectura: Controlador (Maneja la lógica de negocio y controla el flujo del sistema)

Modelos: Cada clase del programa representa una entidad

Vistas: Formularios

Como se estructura el proyecto

El proyecto se estructura en tres carpetas en este caso CData, CNego y Cview

CData:

La carpeta CData contiene la capa de acceso a datos de la aplicación. En ella se agrupan las funciones encargadas de comunicarse directamente con la base de datos, manejar la conexión SQL y ejecutar procedimientos almacenados. Está organizada en varias subcarpetas que cumplen funciones específicas del entorno de desarrollo y ejecución.

Subcarpetas dentro de CData

1 bin

Contiene los archivos **compilados del proyecto** (el resultado final que corre el sistema).

- Dentro de bin se encuentra:

- **net6.0/**: Carpeta donde se guardan los archivos compilados para la versión .NET 6.0.
 - Allí se aloja el archivo ejecutable .dll del proyecto.
 - También pueden encontrarse archivos .pdb (información de depuración).

Esta carpeta se genera automáticamente al compilar el proyecto desde Visual Studio.

2. obj

Almacena archivos temporales de compilación generados por .NET.

- Subcarpetas:
 - Debug o Release (según cómo se compile el proyecto).
 - Cada una tiene una subcarpeta net6.0 con:
 - Archivos .csproj, .nuget.g.targets, .cache, etc.
 - Información necesaria para el proceso de build (compilación).

contiene lógica del programa, pero esta es crucial para que el proyecto se compile correctamente.

3. sql

Aquí se encuentra la lógica real de conexión con la base de datos. Es el corazón de la capa de datos personalizada.

Archivos importantes:

- **ValidadorLogin.cs:**
Clase encargada de verificar el acceso de usuarios mediante el sistema de autenticación.

- **Conexion.cs o ConexionSQL.cs:**

Clase que gestiona la conexión entre la aplicación y la base de datos **Mysql**.

- **ManageSQL.cs:**

Utilidad que interactúa con procedimientos almacenados, realiza consultas y comandos SQL reutilizables.

4. sql/system/

Subcarpeta con archivos administrativos del proyecto:

- **cdata.csproj:**

Archivo de configuración del proyecto. Define:

- Versión de .NET
- Referencias a bibliotecas necesarias
- Dependencias NuGet
- Propiedades de compilación

Este archivo es clave para que Visual Studio sepa cómo compilar esta capa del sistema.

Es decir que:

En la carpeta CData implementa la capa de acceso a datos de la arquitectura MVC.

Contiene la lógica de conexión a la base de datos, validación de usuarios y ejecución de consultas. En ella, se distinguen carpetas como sql, que contiene clases como

ValidadorLogin, ConexionSQL y ManageSQL, las cuales permiten la interacción directa con MySQL mediante procedimientos almacenados. La estructura de compilación y dependencias se encuentra en las carpetas bin, obj y el archivo de proyecto cdata.csproj, que configura el uso de .NET 6.0. Esta organización permite mantener separadas las responsabilidades de compilación, configuración y lógica de acceso a datos.

CNego:

La carpeta CNego implementa la lógica del negocio del sistema dentro del modelo de arquitectura MVC. Esta capa se encarga de recibir los datos desde la vista, procesarlos según las reglas del negocio y, si es necesario, enviar solicitudes a la capa de datos (CData) para guardar, leer o modificar información.

1. Archivos fuente .cs (clases de negocio)

Estas clases representan **entidades funcionales** del sistema, pero con mayor lógica y procesamiento que los modelos puros. No son simples representaciones de datos, sino que **implementan reglas, validaciones, herencia y relaciones funcionales**.

- **CCliente.cs** :Maneja los datos y operaciones relacionadas con los clientes.
- **CFactura.cs**:Representa una factura. Procesa cálculos, validaciones y lógica relacionada.
- **CFactDetalle.cs**:Representa los ítems dentro de una factura. Gestiona la relación factura-productos.
- **Item.cs**:Clase base para artículos en general.
- **CProducto.cs** :Hereda de Item. Agrega propiedades y funciones específicas para productos.
- **CServicio.cs**:Hereda de Item. Especializa funciones para servicios ofrecidos.
- **Persona.cs**:Clase base para entidades tipo persona (clientes, técnicos, etc.).
- **CTecnico.cs**:Hereda de Persona. Gestiona las funciones de técnicos.
- **COrden.cs** :Representa una orden de trabajo, maneja su lógica de estado y relación.
- **COordenDetalle.cs**: Representa el detalle de las órdenes de trabajo

2. Properties/

Contiene archivos auxiliares del proyecto.

- **Resources.Designer.cs:**

Clase generada automáticamente que permite acceder a recursos internos del proyecto (.resx), como textos localizados, íconos, imágenes o cadenas reutilizables.

3. obj/ (igual a CData)

- Contiene carpetas como Debug o Release y subcarpetas net6.0.
- Archivos temporales del compilador:
 - .csproj.nuget.g.targets
 - .json, .cache
 - Archivos de seguimiento de compilación y dependencias

Estos archivos no contienen código funcional, pero son necesarios para compilar el proyecto correctamente.

4. CNegocio.csproj

Archivo de proyecto de C#, donde se definen:

- Versión del SDK (.NET 6.0)
- Dependencias externas
- Configuración de compilación (Debug/Release)
- Ruta a los archivos .cs y recursos del proyecto

Es decir que:

En la carpeta CNego corresponde a la capa de lógica del negocio del sistema, dentro de la arquitectura MVC. En ella se encuentran clases especializadas que representan entidades clave del sistema, como Cliente, Factura, Producto, Servicio u Orden, cada una con funcionalidades específicas más allá de solo almacenar datos. Muchas de estas clases aplican herencia para reutilizar estructuras comunes (por ejemplo, Producto y Servicio heredan de Item; Técnico hereda de Persona). Esta capa se conecta con la vista para recibir acciones del usuario y con la capa de datos (CData) para ejecutar operaciones en la base de datos. Adicionalmente, se incluye el archivo CNegocio.csproj, que contiene las configuraciones necesarias para compilar este proyecto como biblioteca dentro de la solución completa.

Cview

La carpeta CView implementa la capa de presentación del sistema dentro del patrón de arquitectura MVC. Su función principal es mostrar la interfaz gráfica de usuario (GUI) mediante formularios contruidos con WinForms, permitiendo que los usuarios finales interactúen con el sistema de manera visual y directa.

Contenido y estructura de CView

1. Subcarpeta bin

Contiene los archivos compilados del proyecto. Se genera automáticamente al compilar el sistema en Visual Studio.

- bin/Debug/: contiene el archivo CView.1.0.0.nuspec, que forma parte del sistema de empaquetado de dependencias (NuGet).
- bin/Release/: versión compilada en modo de producción.

Estas carpetas contienen los ejecutables y dependencias necesarias para ejecutar el sistema desde Visual Studio o publicar el proyecto.

2. Subcarpeta obj

Contiene archivos temporales utilizados durante el proceso de compilación del proyecto.

- **Archivos como:**
 - CView.csproj.nuget.g.targets
 - Archivos .json (metadatos del proyecto)
 - Archivos de cache NuGet (nuget.cache)
- **Subcarpetas como:**
 - Debug/net6.0/
 - Release/net6.0/

Estos archivos no deben modificarse manualmente. Son generados automáticamente por el compilador .NET.

3. Subcarpeta Properties

Contiene archivos relacionados con los recursos del sistema, como:

- Resources.Designer.cs:

Clase auxiliar que permite acceder a imágenes, cadenas de texto, iconos y otros elementos visuales que se usan en los formularios.
- Archivos .resx (archivos de recursos visuales y textuales).

Aquí se gestionan todos los recursos gráficos y de texto del sistema, centralizados para facilitar cambios o traducciones.

4. Formularios del sistema (Vista)

Estos archivos representan las pantallas funcionales del sistema. Sus nombres están abreviados, pero corresponden a operaciones específicas:

- **cli** Gestión de clientes.
- **fac** Gestión o emisión de facturas.
- **rd** Posiblemente relacionado a reportes de detalle (por confirmar).
- **pro** Manejo de productos.
- **ser** Administración de servicios.
- **tec** Administración de técnicos.
- **ordfrm2** Órdenes de trabajo (pantalla principal).
- **frm2orddet** Detalle de órdenes.
- **frm2pro, frm2ser, frm2serv, frm2tec** Formularios auxiliares de producto, servicio y técnico.
- **frmLoginEvento** Inicio de sesión (login).
- **frmMenu** Menú principal desde donde se accede a las funciones del sistema.

Cada formulario contiene:

- Interfaz gráfica (.Designer.cs)
- Lógica de interacción (.cs)
- Referencias a imágenes o recursos (Resources)

Es decir que:

En la carpeta CView representa la **capa de Vista** del sistema, encargada de mostrar la interfaz gráfica al usuario final mediante formularios desarrollados en WinForms. Esta carpeta contiene los formularios principales (cli, pro, ser, tec, fac, frmLoginEvento, frmMenu, entre otros), así como los recursos visuales asociados (íconos, imágenes y textos). También incluye carpetas de compilación como bin y obj, y archivos de configuración como CView.csproj y Resources.Designer.cs. La solución global del sistema se administra desde el archivo ProyectoFinal.sln, que enlaza todos los proyectos (CData, CNegocio, CView). Esta estructura permite mantener una clara separación entre la lógica de negocio y la presentación, facilitando la escalabilidad y el mantenimiento del sistema.

Funcionalidad del software

El software desarrollado tiene como finalidad gestionar de forma integral los procesos de clientes, productos, servicios, facturación y órdenes de trabajo. Entre sus funcionalidades principales se incluyen: autenticación de usuarios mediante un formulario de inicio de sesión, mantenimiento de registros de clientes y técnicos, creación y consulta de facturas, administración de productos y servicios, emisión de órdenes de trabajo, y visualización de reportes operativos. Todo esto a través de una interfaz gráfica diseñada en WinForms, conectada a una base de datos MySQL

Funcionalidad de la base de datos

Funcionalidad Descripción Técnica

- Almacenamiento relacional Las entidades (clientes, técnicos, producto) están distribuidas en tablas normalizadas.
- Integridad referencial Uso de claves primarias y foráneas para garantizar consistencia entre registros.
- Procedimientos almacenados Ejecutan operaciones específicas como insertar, modificar, validar usuarios.
- Acceso seguro Login de usuario validado directamente desde base de datos.
- Historial de operaciones Guarda trazabilidad de órdenes, facturas, servicios ejecutados.
- Control de stock y servicios Permite registrar el estado del inventario y los servicios disponibles.

Interacción entre Capas del Sistema y la Base de Datos

La arquitectura sigue el patrón MVC, donde cada capa cumple un rol específico:

- **CView:** capa de interfaz con formularios WinForms.
- **CNegocio:** capa de lógica que gestiona procesos y valida datos.
- **CData:** capa de acceso a base de datos (consulta, inserción, actualización).

Flujo de funcionamiento

1. Usuario (CView)
2. Botón o formulario (ej. "Crear cliente")
3. Método en CNegocio (Cliente.cs)
4. Método en CData (ManageSQL.cs Ejecutar "sp_insertar_cliente")
5. Base de datos ejecuta la operación

Resultado se devuelve a la vista

Funcionalidad de los formularios y botones

En la interfaz gráfica (CView), las funcionalidades están distribuidas en secciones, accesibles desde el menú principal (frmMenu).

Sección: Consulta

Permite visualizar datos existentes. Contiene botones para:

- Clientes
- Técnicos
- Productos
- Servicios
- Órdenes
- Facturas

Cada opción abre un formulario que muestra la lista correspondiente desde la base de datos, utilizando procedimientos almacenados de lectura.

Sección: Crear y Modificar Permite registrar o editar información según el tipo de entidad:

- **Entidad** Funcionalidad disponible
- **Clientes** Agregar nuevo cliente / Modificar datos / Validar existencia
- **Técnicos** Agregar nuevo técnico / Editar datos personales
- **Productos** Crear nuevo producto / Editar / Eliminar / Datos de stock, marca, precio
- **Servicios** Registrar nuevo servicio / Modificar / Usar código de referencia
- **Generar Orden** Asignar una orden a un técnico / Registrar defecto / Equipo
- **Bitácora de Orden** Ver orden de ejecución de trabajos / Seguimiento
- **Generar Facturación** Crea factura en base a lo que el cliente solicitó (productos + servicios)

Funcionalidad específica por sección de datos

- **Clientes** Datos personales del cliente (nombre, cédula, teléfono, dirección, correo).
- **Técnicos** Información técnica del personal asignado a órdenes (nombre, cédula, dirección, teléfono, correo).
- **Productos** Nombre, marca, precio, costo, stock disponible, código de referencia, descripción.
- **Servicios** Descripción del servicio, código de referencia, precio, tipo de trabajo realizado.
- **Órdenes** Cliente asociado, equipo en reparación, defecto reportado, técnico asignado, fecha, prioridad.

- **Facturación** Consolidado de lo solicitado por el cliente con precios y totales calculados.

Lógica de Ejecución en Base de Datos

Desde CData, las clases ejecutan los siguientes procedimientos almacenados:

- **sp_validar_usuario** Autenticación desde el formulario de login.
- **sp_insertar_cliente** Agrega un nuevo cliente a la tabla Clientes.
- **sp_modificar_producto** Edita un producto existente.
- **sp_registrar_orden** Registra una nueva orden vinculada al cliente y técnico asignado.
- **sp_crear_factura** Genera una factura basada en productos y servicios seleccionados.
- **sp_bitacora_orden** Lleva registro del orden de ejecución de las órdenes.
- **sp_consultar_servicios** Lista servicios registrados para visualización.
- **sp_eliminar_producto** Elimina un producto del inventario.

Diagrama caso de usos

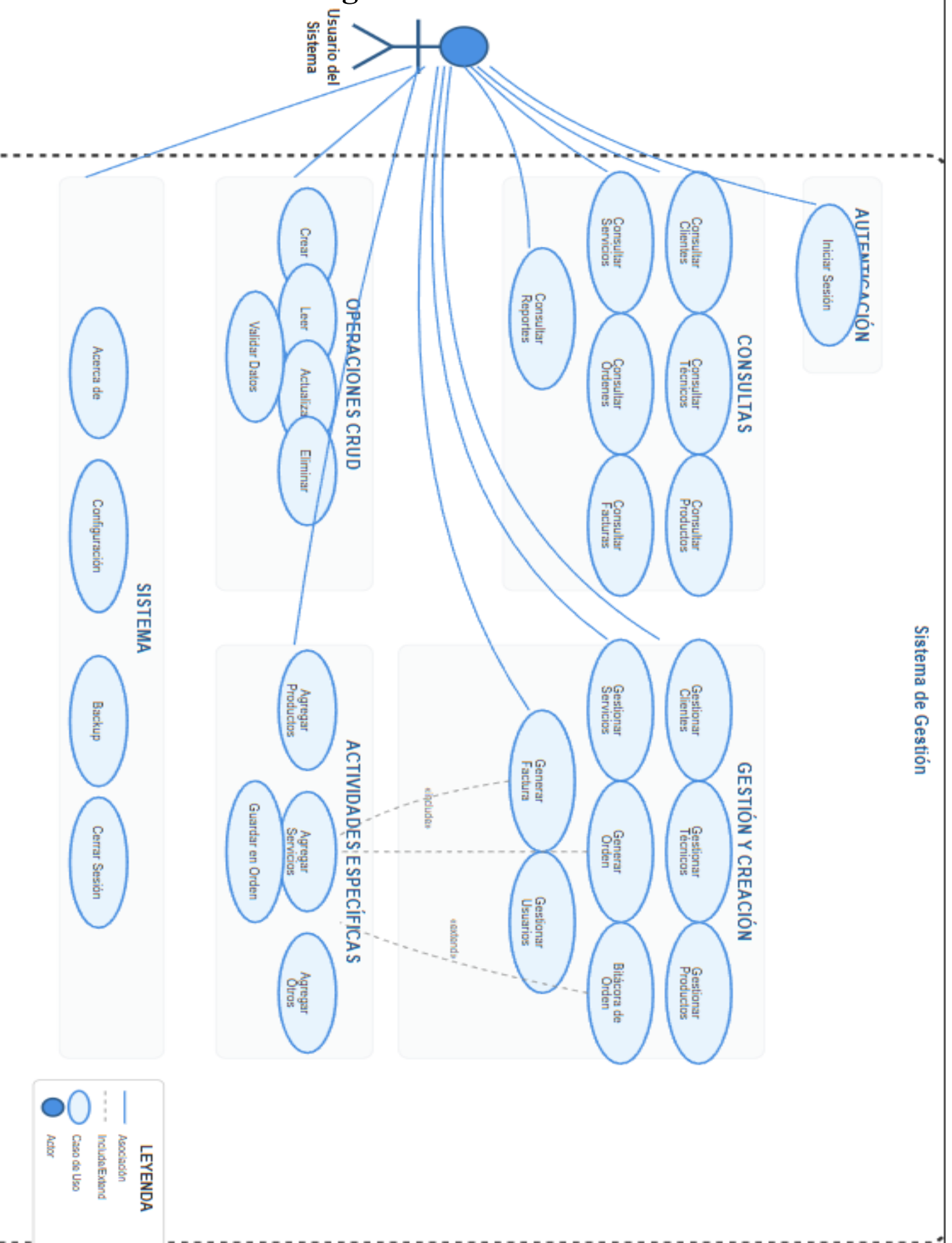


Diagrama entidad relación

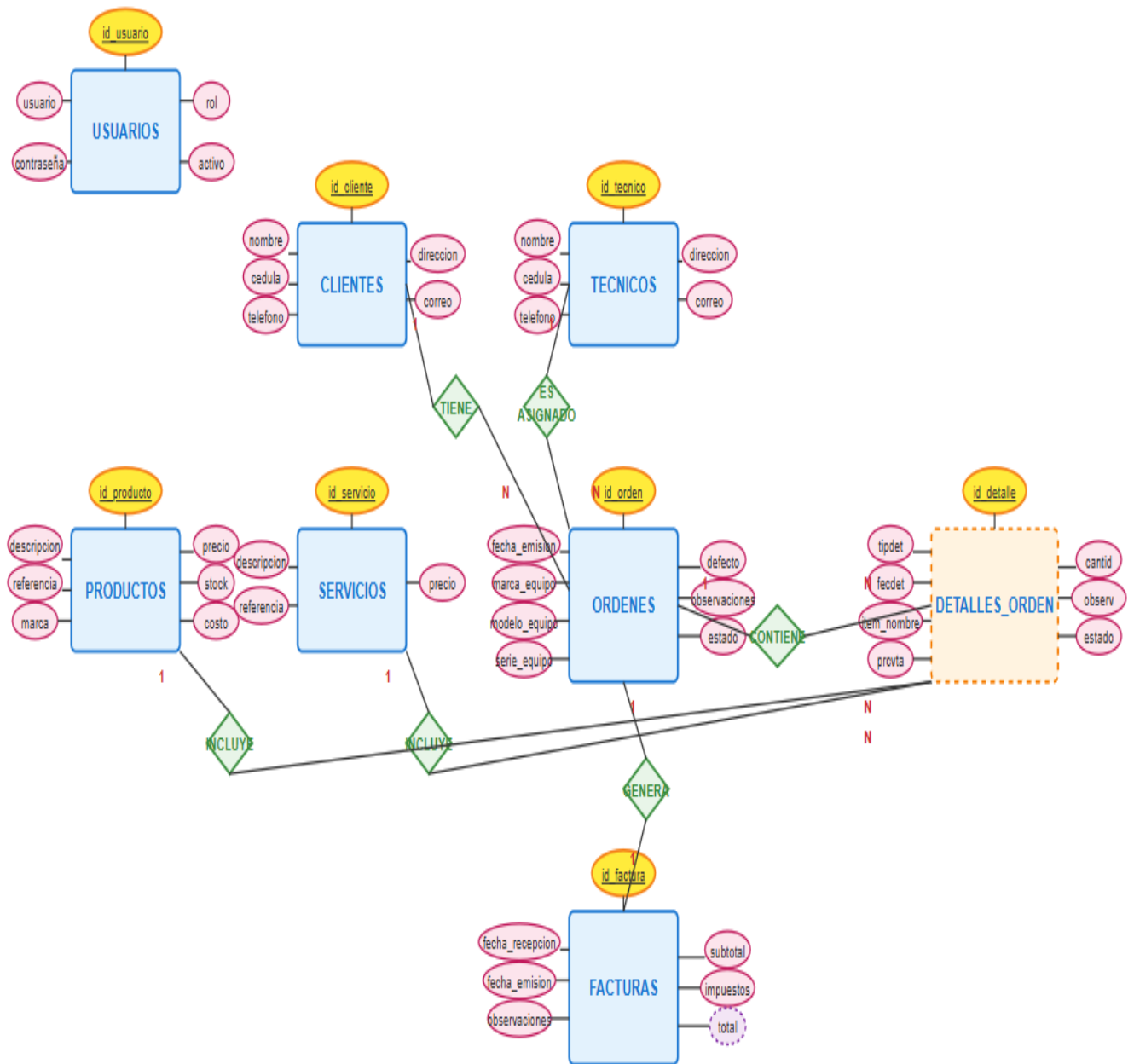
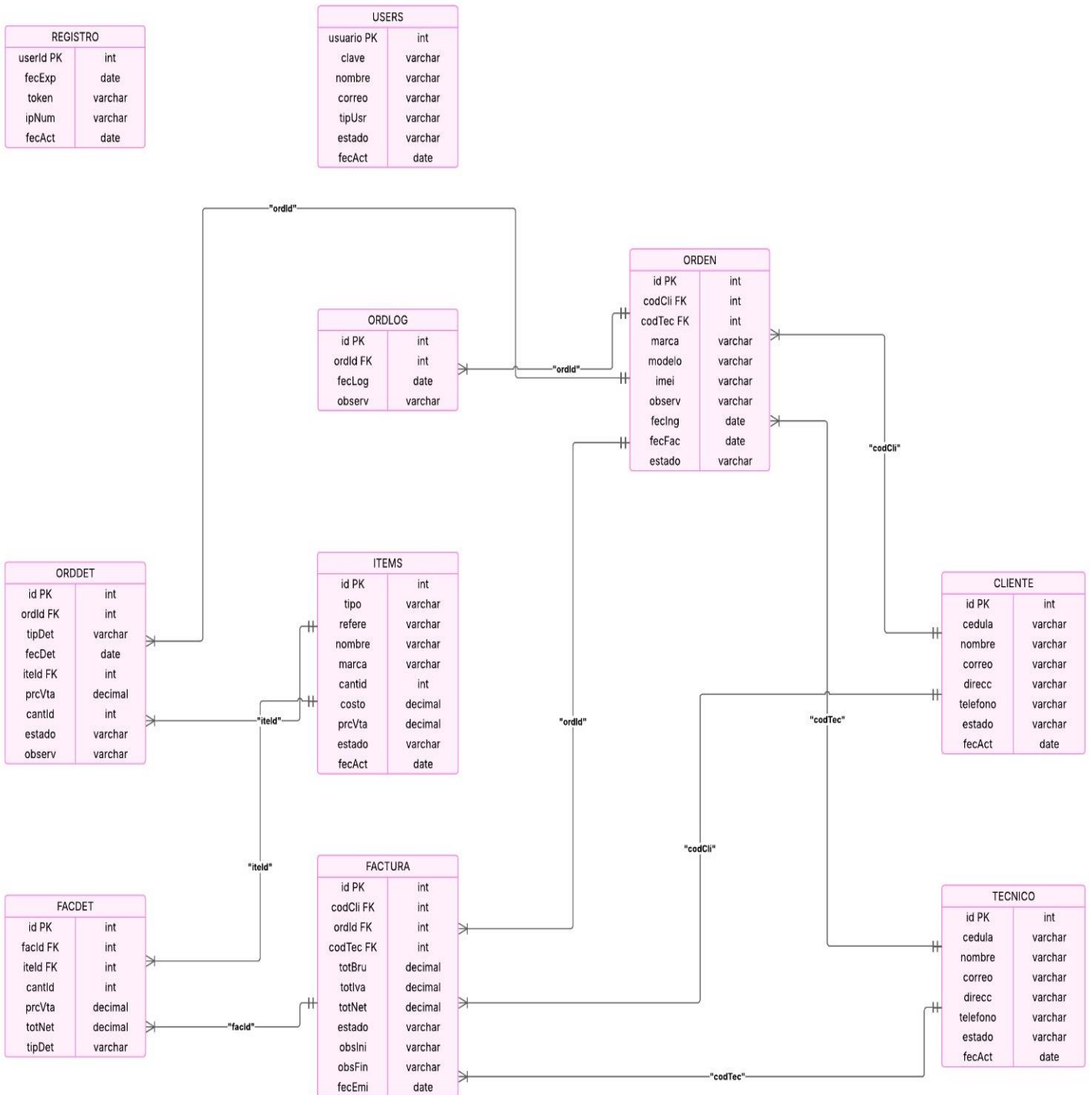


Diagrama Base de datos



Mejoras Proyectadas a Futuro

Aunque el sistema actual cumple correctamente con las funcionalidades requeridas para la gestión de clientes, técnicos, productos, servicios, órdenes y facturación, se identifican varias áreas de mejora que podrían implementarse en futuras versiones para aumentar la eficiencia, la estética visual y la experiencia del usuario.

Mejora de Interfaz Gráfica

- Aplicar un diseño más moderno utilizando librerías visuales avanzadas o migrar de WinForms a WPF (Windows Presentación Foundation).
- Uso de colores coherentes, íconos personalizados y tipografía legible.
- Crear una barra lateral de navegación para facilitar el acceso rápido a secciones.
- Añadir notificaciones (como mensajes emergentes) para confirmar operaciones exitosas.

Generación de Reportes y Exportación

- Incluir generación de reportes en PDF o Excel desde las secciones de facturación, órdenes y bitácoras.
- Implementar filtros de fecha, cliente o técnico para exportar solo lo necesario.

Versión Web o Multiplataforma

- Desarrollar una versión web del sistema usando ASP.NET o Blazor para mayor accesibilidad desde navegadores.
- Posibilidad de una versión móvil para técnicos en campo, que puedan consultar órdenes desde su dispositivo.

MANEJO DE ERRORES

Reporte de Análisis de Excepción

Proyecto: Proyecto_BDA_grupo_2

Fecha: 2025/07/05

Archivo: CData\SQL>LoginDB.cs

Método: ValidarUsuario

Descripción del Error

Tipo de Excepción:

MySql.Data.MySqlClient.MySqlException

Mensaje:

Authentication method 'auth_gssapi_client' not supported by any of the available plugins

Línea:

conn.Open();

Análisis

El error ocurre al intentar abrir una conexión a la base de datos MySQL usando el usuario **root**. El mensaje indica que el método de autenticación configurado en el servidor MySQL (**auth_gssapi_client**) no es compatible con el cliente MySQL utilizado en la aplicación .NET.

Causa Raíz

El usuario **root** en el servidor MySQL está configurado para usar el plugin de autenticación **auth_gssapi_client**, el cual no es soportado por el paquete **MySql.Data** utilizado en el proyecto. Esto genera un fallo inmediato al intentar autenticar la conexión.

Recomendaciones

1.Verificar el plugin de autenticación del usuario en MySQL: Ejecutar en el cliente MySQL:

```
SELECT user, host, plugin FROM mysql.user WHERE user = 'root';
```

2.Cambiar el plugin de autenticación a uno compatible: Por ejemplo, para **mysql_native_password**:

```
ALTER USER 'root'@'localhost' IDENTIFIED WITH 'mysql_native_password' BY  
'Mye-x100pre';
```

```
FLUSH PRIVILEGES;
```

3.Actualizar el cliente MySQL en el proyecto: Asegurarse de usar la última versión de **MySql.Data**.

Información Adicional Solicitada

- Versión del servidor MySQL.
- Plugin de autenticación actual del usuario **root**.

Conclusión:

El error se debe a una incompatibilidad entre el método de autenticación configurado en el servidor MySQL y el cliente utilizado en la aplicación. Ajustando el plugin de autenticación del usuario o actualizando el cliente, se puede resolver el problema.

Reporte de Error: MySql.Data.MySqlClient.MySqlException

Descripción del Error

Durante el proceso de autenticación en el formulario de inicio de sesión (frmLogin), se produjo la siguiente excepción:

Tipo de excepción:

MySql.Data.MySqlClient.MySqlException

Mensaje:

Authentication method 'auth_gssapi_client' not supported by any of the available plugins

Ubicación:

Método: CData.SQL.LoginDB.ValidarUsuario

Archivo: CData\SQL\LoginDB.cs, línea 20

Análisis Técnico

El método ValidarUsuario intenta abrir una conexión a la base de datos MySQL utilizando la clase **MySqlConnection**. Al hacerlo, el cliente intenta autenticarse usando el método **auth_gssapi_client**, el cual no es soportado por el servidor MySQL o por los plugins disponibles en el cliente.

Esto suele deberse a una incompatibilidad entre el método de autenticación configurado en el servidor MySQL y los métodos soportados por la librería cliente utilizada en la aplicación .NET.

Causas Probables

- El servidor MySQL está configurado para usar un método de autenticación no soportado por el cliente.
- La cadena de conexión no especifica un método de autenticación compatible.
- La versión del conector MySQL utilizada no soporta el método requerido.

Recomendaciones

1.Actualizar el conector MySQL a la última versión disponible.

2.Modificar la cadena de conexión para especificar un método de autenticación compatible, por ejemplo:

AuthenticationMethod=mysql_native_password

3.Verificar la configuración del usuario en el servidor MySQL y asegurarse de que utilice un plugin de autenticación soportado, como **mysql_native_password** o **caching_sha2_password**.

4.Consultar la documentación oficial de MySQL y del conector .NET para más detalles sobre compatibilidad de métodos de autenticación.

